

Understanding Trajectron++: A Per-Trajectory Performance Analysis

Simon Drauz and Zoe Matrullo
May 13, 2026

Showcase
Figure

- 1 Motivation
- 2 Analysis Pipeline
- 3 Data Preparation
- 4 Interpretable Modeling
- 5 Feature Effects & Importance
- 6 Performance Regimes
- 7 Model Settings Analysis
- 8 Conclusions

- 1 Motivation
- 2 Analysis Pipeline
- 3 Data Preparation
- 4 Interpretable Modeling
- 5 Feature Effects & Importance
- 6 Performance Regimes
- 7 Model Settings Analysis
- 8 Conclusions

Trajectory Prediction in Autonomous Driving



- A single average ADE hides large variation across trajectories
- Some trajectories inherently hard, others trivially easy — but why?
- No understanding of which trajectory or scene characteristics drive per-trajectory prediction error

Three Research Questions:

- ① Which characteristics affect model performance?
- ② Do distinct *performance regimes* (recurring failure/success patterns) exist?
- ③ How do model settings (history window, prediction horizon, attention radius) affect performance?

Outline

- 1 Motivation
- 2 Analysis Pipeline**
- 3 Data Preparation
- 4 Interpretable Modeling
- 5 Feature Effects & Importance
- 6 Performance Regimes
- 7 Model Settings Analysis
- 8 Conclusions

Track 1 — Full Dataset

- Train Trajectron++ → per-trajectory ADE
- join trajectory + scene characteristics
- interpretable models → feature effects
- performance regime clustering
- cluster inspection

Clustering + cluster inspection run only on the full-dataset path.

Track 2 — Mini Dataset (Settings Sweep)

- Cartesian sweep over model settings
- combine runs
- interpretable models
- feature effects with settings as predictors
(trajectory/scene chars as controls)

- Standard training: batch-averaged loss
- Modification: write one row per trajectory (`data_idx`, `scene_path`, `agent_id`, `scene_ts`, `ml_ade`, `ml_fde`, `min_ade_5`, `nll_mean`, ...)
- `data_idx`: stable trajdata identifier used to join evaluation output with computed characteristics
- Identity validation: reconstruction check ensures the join corresponds to the exact same evaluation context (split, temporal window, etc.)

- 1 Motivation
- 2 Analysis Pipeline
- 3 Data Preparation**
- 4 Interpretable Modeling
- 5 Feature Effects & Importance
- 6 Performance Regimes
- 7 Model Settings Analysis
- 8 Conclusions



● ...

Feature Selection via Mutual Information

● ...

Handling Multicollinearity with VIF

● ...

Outline

- 1 Motivation
- 2 Analysis Pipeline
- 3 Data Preparation
- 4 Interpretable Modeling**
- 5 Feature Effects & Importance
- 6 Performance Regimes
- 7 Model Settings Analysis
- 8 Conclusions

GAM (Penalized Splines)

- Directly interpretable smooth effects
- One spline per feature, no interaction terms
- p-values for significance testing

XGBoost

- Captures non-linearities and interactions
- SHAP values for local + global interpretation

- **Problem:** hyperparameter tuning risks overfitting to a particular data split
- **Solution:** nested cross-validation
 - ▶ Outer folds: evaluate generalization of the full tuning procedure
 - ▶ Inner folds: tune hyperparameters on each outer-fold training set
- OOF evaluation: inspect out-of-fold predictions for instability, outlier folds, bias
- **Conclusion:** tuning generalises stably → proceed to full-data tuning + refit

Final models are refit on *all* data with optimal hyperparameters — no held-out test set, because the goal is understanding, not unbiased prediction.

- Candidates: linear GAM, linear GAM on log-transformed target, Gamma GAM
- Selection criterion: lowest RMSE
- ...

Final Model Performance (OOF)

● ...

Outline

- 1 Motivation
- 2 Analysis Pipeline
- 3 Data Preparation
- 4 Interpretable Modeling
- 5 Feature Effects & Importance**
- 6 Performance Regimes
- 7 Model Settings Analysis
- 8 Conclusions

XGBoost (global SHAP)

GAM (p-values of smooth effects)



Feature Effects — Trajectory Characteristics

● ...

Outline

- 1 Motivation
- 2 Analysis Pipeline
- 3 Data Preparation
- 4 Interpretable Modeling
- 5 Feature Effects & Importance
- 6 Performance Regimes**
- 7 Model Settings Analysis
- 8 Conclusions

- ① Divide trajectories into difficulty groups based on `m1_ade` (easy / medium / hard)
- ② Run clustering on trajectory + scene characteristics per group
 - ▶ Option A: original feature space
 - ▶ Option B: UMAP-reduced space (dimensions chosen via trustworthiness score + elbow rule)
- ③ Assess clustering quality with DBCV score
- ④ Export identified clusters (performance regimes)



Regime Inspection — What Drives Error in Each Regime?

● ...

Outline

- 1 Motivation
- 2 Analysis Pipeline
- 3 Data Preparation
- 4 Interpretable Modeling
- 5 Feature Effects & Importance
- 6 Performance Regimes
- 7 Model Settings Analysis**
- 8 Conclusions

- Dataset: nuScenes mini (full dataset too expensive for repeated runs)
- Cartesian sweep over:
 - ▶ `history_sec`
 - ▶ `prediction_sec`
 - ▶ `attention_radius_m`
- Pooled model: model setting variables + trajectory/scene features as control variables
- Feature normalization: setting-agnostic versions to avoid mechanical confounding (e.g. `mean_path_length_per_second` instead of `path_length`)



...

Outline

- 1 Motivation
- 2 Analysis Pipeline
- 3 Data Preparation
- 4 Interpretable Modeling
- 5 Feature Effects & Importance
- 6 Performance Regimes
- 7 Model Settings Analysis
- 8 Conclusions**

- RQ1 — Characteristics driving performance: ...
- RQ2 — Performance regimes: ...
- RQ3 — Model settings: ...

● ...